159.102 Notes

Section 9 : Computer Graphics and Command Prompt

Computer graphics is a huge and fascinating area and includes the production of games, movies and simulations. This section is a very simple introduction to the basics of how 2-dimensional colour images are displayed. Students who wish to go into the field in more depth should consider taking the course 159.235.

Note that New Zealand (and the UK) spell the word as 'colour'. In the USA it is spelled as 'color'.

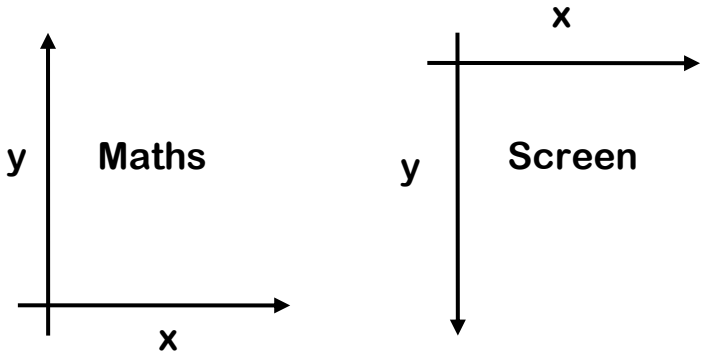
The computer display

The most familiar display is the monitor (or screen). A screen consists of thousands of little dots or **picture elements** (abbreviated to **pixel**). The idea of a "picture element" is thought to have originated in Germany in 1880 as the word *Bildpunkt* or "picture point" which then was translated into English as "picture element".

Every pixel produces one colour (or *hue*) on the screen. Thus a colour image is created by many thousands of pixels, each one set to a particular colour. When displaying an image such as a photo, it is important to use as many pixels as possible to attempt to closely resemble the original. If too few pixels are used, a noticeable "step-effect" will occur along the edges. The number of pixels used for a display is called the **resolution**. High resolution = lots of pixels and low resolution = less pixels.

Most displays use the two-dimensional (x, y) coordinate system – known as a 2D display.

Note that the y axis runs down the screen – unlike maths where the y axis runs upwards.



The resolution of a display is usually stated as the number of pixels in the x-direction followed by the number of pixels in the y-direction. For example, a "1920 x 1080" display means that the screen has 1920 pixels running across it and 1080 pixels running down it for a total of just over 2 million pixels.

In order to display an image on the screen, it is necessary to set every pixel to a particular colour.

The RGB Colour Model

Most computer displays use the RGB model to generate colours on the screen. Each pixel can produce three overlapping beams of light (Red, Green and Blue). Each beam can be set to a particular **intensity**. The intensity of a beam can range from zero (the beam is off) to full intensity and anything in between.

The mixing of the coloured beams (and the settings of the intensities) produces a wide range of colours. For example, if we want a red dot to show on the screen, we would set the beams for that pixel to be Red (full intensity), Green (off) and Blue (off). Intensities are given as an integer value

from zero to 255 where zero means the beam is off and 255 means it is at full intensity. Thus the red pixel in our example would be defined as Red = 255, Green = 0, Blue = 0.

Naturally the maximum value of 255 has been carefully selected so that each intensity value can be stored in 8 bits (unsigned). Thus the definition of any particular colour requires 24 bits (3 bytes) of storage (for the three beams) and a colour image of 2 million pixels requires approximately 6 million bytes (6 MB). Storage space for many high-resolution images can easily become a problem.

It is useful for programmers to have a basic idea of the intensities and combinations needed to produce a particular colour. Useful colours to remember are black is (0, 0, 0) (switch all the beams off) and white is (255, 255, 255) (switch all the beams on full). Fortunately, the other colours are provided (in immense detail) on many websites. Just google "RBG colour chart" or look here:
https://www.rapidtables.com/web/color/RGB_Color.html

Note that many of these charts provide the intensity values in **hexadecimal**. Fortunately, everyone in this class is an expert at hexadecimal so that will not be a problem.

By the way, the RGB system is used to create colours using light. A completely different system (the CMYK colour model) is needed to create colours on paper (using ink, paint, etc). The two systems work completely differently. For example, in RGB all beams on full create white light. With paint, if you mix all paints together, you won't get white. If you get the opportunity, speak to someone who controls the lighting at a theatre – it is a fascinating subject.

If you are interested, have a look at the information on the RGB colour model here:
https://en.wikipedia.org/wiki/RGB_color_model

File Formats for Graphics

As discussed above, an image consists of millions of numbers (intensities) arranged in a particular way. Usually, we want to keep our image stored on a computer and so we put it into a file. There are many standard file formats available for graphics. The most common is JPEG commonly used for photos because they use various cunning tricks to reduce the number of pixels that you need to store. This approach is known as *compression* and works very well for photographs but does not work well when printed details appear in the image, for example closely typed text.

Compression is not part of our course, but if you are interested you can read more about it here and see the effects of too much compression on a brain scan image:
https://en.wikipedia.org/wiki/JPEG

By the way (as shown in this article) compression algorithms are a good example of the answer to the question "is maths important for Computer Science?". Just a reminder, JPEG and compression are not part of our course.

So people use a number of different file formats for different reasons. Some formats (like JPEG) reduce the amount of storage needed but can also lose detail. Other formats retain all the detail but need lots of storage. You select what you need for the project you are working on.

Several common file formats are listed here (and are not part of our course):
https://blog.hubspot.com/insiders/different-types-of-image-files

We are going to use a format called Portable Pixel Map (PPM) for the simple reason that it is the easiest format for programmers. Unfortunately the PPM format is almost unheard of – like most things that are designed to assist programmers. When we create a PPM file we don't have any way of

displaying the image because most standard "image displayer" apps do not recognise PPM. Weird that they don't recognise the format that is most useful for programmers but there you are.

There are two ways to display a PPM image: (a) search the internet for an app that displays PPM (good luck with that option) or (b) convert your PPM image to a well-known format (such as BMP) and display that. If you convert to JPEG, expect to lose some detail from your image – it may be interesting to try this.

The PPM File Format

A PPM is a text file (that's useful because we know how to use text files) that has no encoding or compression and consists of normal ASCII codes (all good news for us). A PPM file is divided into two sections – the header information and the RGB data.

The header information looks like this:

```
P3
# myimage.ppm
500 300
256
```

Notes about the header information:

Line 1 must have P3 on it. Nobody knows why.

Line 2 has a hash (#), then a space and then the name of your file (with the extension .ppm)

Line 3 has the number of pixels in the x direction (500 in this example) and then a space and the number of pixels in the y direction (300 in this example).

Line 4 has one number which tells you how many intensity levels are being used. We use the standard system of 0 to 255 which means that a total of 256 intensity levels are available.

The header information is followed (on the next line) by the RGB data.

Each line of RGB data looks like this:

```
255 153 0  255 153 0  255 153 0  255 153 0  255 153 0
```

Each set of three numbers is the RGB values for one pixel. Remember the order, first R then G and finally B. So in this example, the first pixel has the values Red = 255, Green 153, Blue = 0. Note that there is one space between the numbers. There are two spaces at the end of the set of values for one pixel and then we move on to the next pixel (which in this example is the same).

The RGB values must be written to the text file (using this format) for one row of pixels across the display (starting at the top). Then you must end the line – output `endl` to the file. Then move on to the next row of pixels, and so on.

In this example we are displaying 5 pixels in a row across the display starting at the top left corner. Each pixel is orange (that's what you get with 255 153 0). This image is so small you would struggle to see it.

Data structure for a graphics program

An image is a collection of pixels where each pixel has three integer values (R, G and B). If you want to save space you could make them unsigned char (8 bits). But we actually are going to write the data to a text file so it doesn't matter if it is initially stored as an integer or as unsigned char. Here is the class we need for a pixel:

```
class pixel_class {
private:
  int red, green, blue;
public:
  void loaddata(int v1, int v2, int v3);
  void datatofile(fstream & ppmfile);
  int getR() { return red; }
  int getG() { return green; }
  int getB() { return blue; }
};

void pixel_class::loaddata(int v1, int v2, int v3) {
  red = v1;
  green = v2;
  blue = v3;
}

void pixel_class::datatofile(fstream & ppmfile) {
  // write the RGB values for one pixel to the ppm file
  ppmfile << red << " " << green;
  ppmfile << " " << blue << "  ";
}
```

Important points about this code:

- a pixel consists of 3 integer values called red, green and blue
- we have some useful methods to 'set' and 'get' the RGB values
- an `fstream` parameter must be **call-by-reference** (note the &)
- this is a very useful class and will work for any 2D image
- advanced programmers don't need a program – we can discuss sections of code

Setting up the picture array

An image is a two-dimensional array of pixels. We usually know exactly how many pixels there across the image (the x-axis) and how many pixels there are down the image (the y-axis). We can then set up a 2D array of pixels like this:

```
pixel_class picture[250][100];
```

This will create an image with 250 pixels across the screen and 100 pixels down the screen. A 2D array has two sets of square brackets. When you want to access a particular pixel you need to show one set of square brackets for the x-value and another set for the y-value, like this:

```
picture[x][y].loaddata(255, 153, 0);
```

We are now ready to write a full program.

Program 1 : create an image consisting of an orange rectangle

Things to do before you start:

- decide how big your image will be – I have selected 250 pixels across and 100 pixels down
- find the RGB values for orange (use an RGB website) – they are 255, 153, 0

And then write the program:

```
#include <iostream>
#include <fstream>
using namespace std;

class pixel_class {
private:
  int red, green, blue;
public:
  void loaddata(int v1, int v2, int v3);
  void datatofile(fstream & ppmfile);
  int getR() { return red; }
  int getG() { return green; }
  int getB() { return blue; }
};

void createpicture();

string filename;
pixel_class picture[250][100];

int main() {
  int x, y;
  fstream myfile;
  createpicture();
  filename = "myImage.ppm";
  myfile.open(filename.c_str(), fstream::out);
  myfile << "P3\n";
  myfile << "# " << filename << endl;
  myfile << 250 << " " << 100 << endl;
  myfile << 256 << endl;
  for (y = 0; y < 100; y++) {
    for (x = 0; x < 250; x++) {
      picture[x][y].datatofile(myfile);
    }
    myfile << endl;
  }
  myfile.close();
}

void createpicture() {
  int x, y;
  for (y = 0; y < 100; y++) {
    for (x = 0; x < 250; x++) {
      picture[x][y].loaddata(255, 153, 0);
    }
  }
}
```

```
//--------------- methods for the pixel_class ------------
void pixel_class::loaddata(int v1, int v2, int v3) {
  red = v1;
  green = v2;
  blue = v3;
}

void pixel_class::datatofile(fstream & ppmfile) {
  // write the data for one pixel to the ppm file
  ppmfile << red << " " << green;
  ppmfile << " " << blue << "  ";
}
```

Note the double for loop used for the 2D array:

```
  for (y = 0; y < 100; y++) {
    for (x = 0; x < 250; x++) {
      picture[x][y].datatofile(myfile);
    }
    myfile << endl;
  }
```

Note how the main program outputs the header information to the .ppm file (as defined earlier) including the size of the image which, in this example, is 250 x 100 pixels.

Remember that the inner loop will run many times for each value of the outer loop. Thus y is set to 0 and x runs all the way from 0 to 249. This is the first row across the image. All these pixels have the same y value (of zero) but they all have a different x value (from 0 to 249). Then y changes to 1 and x runs all the way from 0 to 249. This is the second row of pixels across the image. And so on until y gets to 99 and x runs all the way from 0 to 249 which will be the last row of pixels across the image.

The actual image is created in the function `createpicture`. We load each pixel with appropriate RGB values. In this case, we are creating a rectangle that is uniformly orange. If you wanted an image of the sea and the beach and the sky, then you would need a lot more code and you be assigning a range of different RGB values to different pixels.

Viewing your image

If all has gone according to plan, your program (above) will have created a new image stored in a file called `myImage.ppm` – you can use whatever filename you like. If you look in the folder where the program was running, you should see this file in there.

In an ideal world, you could now double-click on the image file and you would see your image. Unfortunately, for most of us, when you double-click on a .ppm file your system will say "I don't know what a .ppm file is – please find an app that can display this image." (or words to that effect).

At this stage you have two options:

a) search the internet for an app that can display a .ppm file
b) use the internet to convert your .ppm file to a format that most viewers recognise (e.g. .bmp)

The easiest option is (b). Search the internet for a graphics format converter. For example:
https://convertio.co/ppm-bmp/

Once you have converted your ppm file to a bmp file you should be able to double-click on the .bmp file and admire your orange rectangle.

Program 2 : create a more exciting image

Things to do before you start:

- decide how to change your image – I decided to include a vertical line in purple
- find the RGB values for purple (use an RGB website) – they are 153, 51, 255

Here is a useful trick to create a more complicated image – work in "layers of colour". For example, to create an image of an orange rectangle with a vertical purple line:
Step 1: make a completely orange rectangle
Step 2: change selected pixels to be a different colour (e.g. purple)

Here are the additions to the program – the rest is the same as Program 1 above:

in the main program:

```
  createpicture();
  purpleline();
```

So I create an orange rectangle and then (new function) place a purple line on top of it.

```
void purpleline() {
// vertical line (one pixel wide) at position x = 50
  int y;
  for (y = 0; y < 100; y++) {
    picture[50][y].loaddata(153, 51, 255);
  }
}
```

Program 3 : create an image of a flag

Flags are very much part of the real world. There are flags all over the place, for example the Olympic games. Flags are interesting images to work with and some of them are challenging to create.

Things to do before you start:

- find out more about flags (especially the *aspect ratio* – height vs length)
- decide which flag you want to depict
- look up specific details (size, shape, colour, etc) for that particular flag

If you are creating an image of a flag for an official occasion, you can face heavy a heavy fine if you do not depict the flag correctly! You need to get the shape and the colours exactly right.

I have decided to create an image of the flag of Ireland. I discover that the flag consists of 3 vertical blocks of colour. The first block is green, the second block is white and the third block is orange.

I then look up the *aspect ratio* for the flag here:
https://en.wikipedia.org/wiki/List_of_aspect_ratios_of_national_flags

I discover that the aspect ratio for the flag of Ireland is 1:2. For every 1 pixel down the screen (y) I need 2 pixels across the screen (x). Decide on the size of your image using this aspect ratio – I will decide 500 x 250.

Next, I look up the correct RGB values for the colours in the flag here:
https://flagcolor.com/irish-flag-colors/

I discover that the RGB values for the correct shade of green are 22, 155, 98
I discover that the RGB values for the correct shade of orange are 255, 130, 0
And white is... white which is 255, 255, 255 (all beams on full)

I copy my program from above (to make use of the pixel class and the main program) and then I include a new function to create the image of the flag. My program looks like this:

```
#include <iostream>
#include <fstream>
using namespace std;

class pixel_class {
private:
  int red, green, blue;
public:
  void loaddata(int v1, int v2, int v3);
  void datatofile(fstream & ppmfile);
  int getR() { return red; }
  int getG() { return green; }
  int getB() { return blue; }
};

void IrelandFlag();

string filename;
pixel_class picture[500][250];

int main() {
  int x, y;
  fstream myfile;
  IrelandFlag();
  filename = "Flag.ppm";
  myfile.open(filename.c_str(), fstream::out);
  myfile << "P3\n";
  myfile << "# " << filename << endl;
  myfile << 500 << " " << 250 << endl;
  myfile << 256 << endl;
  for (y = 0; y < 250; y++) {
    for (x = 0; x < 500; x++) {
      picture[x][y].datatofile(myfile);
    }
    myfile << endl;
  }
  myfile.close();
}

void IrelandFlag() {
  int x, y;
  // green block up to one third of 500
  for (y = 0; y < 250; y++) {
    for (x = 0; x < 167; x++) {
```

```
      picture[x][y].loaddata(22, 155, 98);
    }
  }
  // white block from one third to two thirds of 500
  for (y = 0; y < 250; y++) {
    for (x = 167; x < 333; x++) {
      picture[x][y].loaddata(255, 255, 255);
    }
  }
  // orange block from two thirds of 500 to the end
  for (y = 0; y < 250; y++) {
    for (x = 333; x < 500; x++) {
      picture[x][y].loaddata(255, 130, 0);
    }
  }
}

//--------------- methods for the pixel_class ------------
void pixel_class::loaddata(int v1, int v2, int v3) {
  red = v1;
  green = v2;
  blue = v3;
}

void pixel_class::datatofile(fstream & ppmfile) {
  // write the data for one pixel to the ppm file
  ppmfile << red << " " << green;
  ppmfile << " " << blue << "  ";
}
```

I can now run my program, create the file called Flag.ppm, convert it to Flag.bmp and then admire my image.

It is very easy to change my program to create an image of any other flag. Simply set the size (based on the aspect ratio) and then make a new function (in place of `IrelandFlag`) that sets the specified pixels to the correct RGB values.

If you feel sufficiently challenged to attempt to draw the New Zealand flag (this is not easy!) then there are three things to keep in mind:

- remember to use layers of colours, first completely blue, and so on.
- don't just use any old shade of blue – find the correct RGB vales
- get all the proportions right by checking here: https://mch.govt.nz/nz-identity-heritage/flags/description-and-dimensions

Using the command prompt (*this is a separate section from graphics*)

The command prompt is an application that simulates what all computer screens looked like before 1995 (or so). On a Windows computer you can get the command prompt by selecting Start (the window symbol in the bottom corner of your screen) and typing command prompt. There are also some other ways to run the command prompt. The command prompt is also known as *terminal*. For a Mac, you will need to google how to use the command prompt or watch this: https://www.youtube.com/watch?v=PlUHZc7VOME

When you run the command prompt it starts a window which is black and empty with maybe a little bit of text in it. Take a moment to reflect on the fact that before 1995, this is what every computer screen looked like – most of the time.

You should also notice that when you start running one of your C++ programs, you are actually using the command prompt screen, i.e. a black window with some text displayed in it. There are no colours, no windows, etc in the command prompt window.

The important thing about the command prompt is this – while the display may have changed since 1995, the computer still functions using the command prompt screen. When you start up your computer and you see windows and colours (etc) you are actually looking at the display of somebody else's program. That program may be called Windows, for example.

The command prompt allows you to enter *system commands* in order to navigate around your system, execute programs, etc. Again, before 1995 (or so) this was the only way to do anything useful on a computer.

A list of system commands for Windows (known as DOS commands) appears here:
https://en.wikipedia.org/wiki/List_of_DOS_commands

A list of system commands for Mac (known as Mac Terminal commands) appears here:
https://www.makeuseof.com/tag/mac-terminal-commands-cheat-sheet/

The most useful command (in both systems) is cd to "change directory".
`cd datafolder` will move you to the folder called `datafolder`
`cd..` will move you up one folder in the hierarchy of folders

Another useful command is `dir` (Windows) or `ls` (Mac). These commands give you a listing of the files in the current folder (or directory). If you navigate to a particular directory (or folder) by using `cd`, you can then enter `dir` (or `ls`) and see a list of all files in that particular folder.

You can execute machine code programs (.exe files) as long as you are in the directory containing the .exe file. Simply enter the name of the .exe file at the command prompt and press Enter. (Always press Enter after typing a system command). For example, if you wrote a program and stored it in a file called prog1.cpp and you then successfully compiled your program (translated it to machine code), the system would create a new file called prog1.exe. To run your program, you enter the command prompt screen, enter prog1 (the name of your program) and press enter. The program will run – try it out with one of your old programs.

You can also make your program execute system commands while it is running. For example, you could make your program carry out the `dir` (or `ls`) command and the program would display the files in the current directory while your program is running. You do this by using the `system` function in the `cstdlib` library. Here is a program using the `system` function to display all files in the current directory. Note that the system command is a string inside double quotes:

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main() {
  system("dir");
}
```

WARNING – this program may not run on all systems.

Many programmers like to use the system command `cls` (Windows) or `clear` (Mac) to clear the screen during the running of a program. Here is a program using this command:

```
#include <iostream>
#include <cstdlib>  // for system
#include <cstdio>   // for getchar
using namespace std;

int main() {
  cout << "This is page one of the display...\n\n";
  cout << "some stuff on page one... \n\n\n\n";
  cout << "Press Enter to display page two";
  getchar();
  system("cls");
  cout << "This is page two of the display...\n\n";
  cout << "some stuff on page two... \n\n\n\n";
}
```

WARNING – this program may not run on all systems.

The ***knowledge*** about the command prompt is part of our course. If you continue with Computer Science, some of the higher-level courses may assume a basic familiarity with the command prompt and some of the available commands.

However, in 159.102 you are ***not required*** to use the command prompt. This is because the system commands differ from one system to another and some systems may not be able to use the system commands at all.

If the programs shown above (using the system function) do work on your computer, I encourage you to explore using the commands available with the command prompt.

Redirection of output when using system commands

When you use a system command such as `dir` (or `ls`) it creates a display. You can *redirect* the output to a text file by using the > symbol (on some systems the >> symbol). When the output is redirected, no display appears and all the text of the display is sent to the text file instead.

If I entered the command `dir` (in Windows) I would get a display listing the file in my directory.

But if I redirected the output like this: `dir > myfile.txt` the display would not appear on the screen. All the lines of the display would be put into the file called myfile.txt and this file would appear in the directory that I am in when I use the system command.

I can also use redirection from inside my own programs using the `system` function like this:

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main() {
```

```
  system("dir > myfile.txt");
}
```

WARNING – this program may not run on all systems.

After this program has been executed, a new text file called myfile.txt will be available in the folder where you ran the program. Inside this file you will find all the lines of text that would otherwise have been displayed on the screen.

Once again, the ***knowledge*** of redirection of output is part of our course.

However, in 159.102 you are ***not required*** to use redirection of output. This is because the system commands differ from one system to another and some systems may not be able to use the system commands at all.

Redirection of output when running your own programs

This is an optional section and is not required for 159.102

If you write a program that uses the cout command, it will display text on the screen. However, if you run your program using the command prompt, you can redirect the output to a text file. This can be useful if you are testing your program and you want to keep your display to check later.

The steps required are as follows:

1. find an old program that displays something on the screen – assume it is called prog1.cpp
2. run the program and check that the compiler (Atom or similar) has created prog1.exe
3. make a note of which folder contains the file called prog1.exe (or move it to a folder)
4. start the command prompt
5. navigate to the correct folder
6. enter the command `prog1 > myfile.txt`
   (*i.e. the name of the program (without the .exe) and then the name of a text file*)

The output from your program will now be in the file called myfile.txt.

Once again, redirection of output from your own program is not part of our course and is for interest only.